

FINAL REPORT

AUTOMOTIVE INSURANCE CLAIMS RISK TRIAGE

Mriganko Chowdhury¹, Aryan Gonsalves¹, Ashish Raj Singh¹, Deborah Sheryl Veluvalli¹, Kshama Girish¹

¹Arizona State University, Tempe, Arizona, USA

ABSTRACT

This project develops an end-to-end fraud triage system for automotive insurance claims and evaluates its generalization on a Medicare provider-fraud dataset. The automotive dataset contains 15,420 claims (5.99% fraud), while the Medicare dataset aggregates 558,211 records into 5,410 providers (9.35% fraud). The pipeline performs schema validation, engineers 41 features (including guideline-based risk flags), and trains a tuned XGBoost model alongside CatBoost and a stacking ensemble.

Predictions are explained using SHAP reason codes and paired with retrieval-augmented (RAG) triage briefs grounded in fraud-guideline documents. Claims are routed into SIU Escalation, Manual Review, or Approve buckets. The automotive model achieves Test PR-AUC 0.2443 with 5.4× enrichment in the top 5%, while the Medicare model reaches PR-AUC 0.6869 with 8.9× enrichment.

The system is deployed live on Hugging Face Spaces as an interactive dashboard, containerized, tracked with MLflow, and audited for fairness using the 80% disparate-impact rule.

Keywords: insurance fraud detection, risk-based triage, SHAP explainability, gradient boosting, CatBoost, Optuna optimization, retrieval-augmented generation, decision support systems, fraud risk scoring

NOMENCLATURE

Acronyms

CI/CD	Continuous Integration / Continuous Deployment
DI	Disparate Impact (ratio)
KS	Kolmogorov–Smirnov (two-sample test)
LLM	Large Language Model
OHE	One-Hot Encoding
PR-AUC	Precision–Recall Area Under the Curve
RAG	Retrieval-Augmented Generation
ROI	Return On Investment
SHAP	SHapley Additive exPlanations
SIU	Special Investigations Unit
TPE	Tree-structured Parzen Estimator
XGBoost	eXtreme Gradient Boosting

Symbols

N	Total number of claims in a dataset
$y_i \in \{0, 1\}$	Ground-truth fraud label for claim i
$\hat{p}_i$	Predicted fraud probability for claim i
k	Top- k ranking threshold (e.g., $k = 5\%$)
$\tau_{\text{SIU}}, \tau_{\text{MR}}$	Triage cut-points (top-5%, next-15%)
ϕ_{ij}	SHAP value of feature j for claim i
DI_g	Disparate-impact ratio for demographic group g

1. INTRODUCTION

Insurance fraud is a costly and persistent problem. The FBI estimates non-health insurance fraud at around \$40 billion annually in the United States, raising household premiums by several hundred dollars [1]. In automotive insurance, fraud is rare per claim but expensive in aggregate, making the key challenge how to allocate limited investigation resources. Since reviewing every claim is infeasible and approving all claims is unacceptable, this project applies *triage*: ranking claims by risk and routing only the highest-risk subset to investigators.

This project builds an end-to-end triage system and evaluates whether the methodology transfers across fraud domains. The design is guided by five requirements: minority-class performance using PR-AUC and Precision@5%, explainability for SIU reviewers, policy-grounded outputs via fraud-guideline briefs, fairness assessment using the 80% disparate-impact rule, and production readiness through schema validation, drift monitoring, automated testing, MLflow tracking, and one-command execution.

Developed by Team Connecticut for the FSE 570 Capstone at Arizona State University, the system uses two datasets. The primary dataset (fraud_oracle.csv) contains 15,420 claims and 33 attributes, with 923 fraud cases (5.99%) [2]. A secondary Medicare provider-fraud dataset provides a structurally different setting for cross-domain evaluation [3]. A curated fraud-guidelines corpus supports retrieval-augmented decision-making by grounding outputs in procedural guidance.

The primary dataset is split into 80/10/10 stratified training, validation, and test sets (12,336/1,542/1,542 claims). The modular pipeline includes validation, preprocessing, feature engineering, modeling, explainability, RAG brief generation, fairness

analysis, monitoring, temporal analysis, and dashboard components, all executable via `python train.py` with MLflow tracking.

A key finding was the removal of `PolicyNumber`, a near-unique identifier (15,420 unique values). Its inclusion caused severe leakage, inflating Validation PR-AUC from 0.2518 to 0.7166 and Test PR-AUC from 0.1951 to 0.7926. All results therefore use the corrected leakage-free pipeline.

The remainder of the report covers data preparation, system architecture, methodology, results, limitations, deployment recommendations, and conclusions.

2. DATA SOURCES AND PREPROCESSING

2.1. Dataset Selection

The primary dataset is the Vehicle Insurance Fraud Detection corpus from Kaggle (`fraud_oracle.csv`) [2], containing 15,420 claims and 33 attributes covering policy, claim, vehicle, and demographic details. The target `FraudFound_P` flags 923 fraudulent cases (5.99%), reflecting realistic class imbalance in automotive insurance.

The secondary dataset is the Healthcare Provider Fraud Detection dataset [3], provided as four CSVs (Inpatient, Outpatient, Beneficiary, Provider) with 558,211 claim records. These are joined on Provider ID and aggregated into a 5,410-provider feature matrix, where `PotentialFraud` flags 506 providers (9.35%). The two datasets differ in domain, schema, and aggregation level, enabling cross-domain evaluation.

A third resource is a curated fraud-guidelines corpus consisting of four markdown documents (`fraud_red_flags.md`, `triage_procedures.md`, `policy_coverage_standards.md`, `staged_accident_patterns.md`). These are not used for model training but are embedded and stored in ChromaDB to support RAG-based triage briefs (Section 4). They also inform feature engineering, with several risk flags directly derived from guideline rules.

Overall, the two datasets provide a deliberate cross-domain comparison. The automotive dataset represents claim-level fraud detection, where each row is a single insurance claim, while the Medicare dataset represents provider-level fraud detection, where many claims are aggregated to one provider. This difference in domain, scale, schema, and unit of analysis allows the project to test whether the same triage methodology generalizes beyond one dataset.

2.2. Data Cleaning and Preprocessing

The automotive dataset is relatively clean but requires targeted preprocessing. The pipeline in `data_preprocessing.py` removes identifier-like columns via `COLS_TO_DROP` (notably `PolicyNumber`, with 15,420 unique values, to prevent leakage), imputes sentinel values (e.g., `Age = 0`) using median age bands, and standardizes categorical labels. The data is then split into 80/10/10 stratified training, validation, and test sets, preserving the 5.99% fraud rate (12,336 / 1,542 / 1,542 claims).

The Medicare dataset requires more extensive preprocessing due to its multi-table structure. Inpatient, Outpatient, and Beneficiary tables are joined to the Provider table, and claim-

level variables (e.g., reimbursement, deductible) are aggregated using sums and averages. Additional features such as distinct beneficiary and physician counts and average claim value are engineered. Missing values after aggregation are set to zero, producing a $5,410 \times 22$ provider-level feature matrix with `PotentialFraud` as the target.

The sklearn preprocessing pipeline [4] is fit on the training split and applied unchanged to validation and test data. Numeric features are median-imputed and standardized, while categorical features are mode-imputed and one-hot encoded. The fitted preprocessor is serialized to ensure consistent transformations during inference.

2.3. Feature Engineering and Risk Taxonomy

Raw model performance on the 33 original columns is limited because many of the most informative variables are stored as ordinal strings (e.g., `PastNumberOfClaims` as "none", "1", "2 to 4", "more than 4") or as raw categorical values that miss higher-order interactions known to be predictive of fraud. The feature engineering module `feature_engineering.py` addresses this by adding 41 engineered features on top of the original schema. These features fall into six groups summarized in Table 1.

TABLE 1: Engineered feature taxonomy (41 features in total).

Group	Count	Examples
Ordinal $\rightarrow$ numeric	13	<code>PastNumberOfClaims_Num</code> , <code>AgeOfVehicle_Num</code> , <code>VehiclePrice_Num</code>
Time-based	3	<code>WeekGap</code> , <code>MonthGap</code> , <code>SameDayOfWeek</code>
Binary risk indicators	11	<code>NoPoliceReport</code> , <code>NoWitness</code> , <code>PolicyHolderFault</code> , <code>YoungDriver</code> , <code>HighDeductible</code> , <code>WeekendAccident</code> , <code>WeekendClaim</code>
Interaction / ratio	6	<code>Age_x_Deductible</code> , <code>Fault_NoPolice</code> , <code>Price-ToDeductible</code>
Guideline-grounded flags	7	<code>Liability_NoPolice</code> , <code>HighValue_Liability</code> , <code>Sport_YoungDriver</code> , <code>VeryEarlyClaimFlag</code>
Composite	1	<code>RiskFlagCount</code>

The most distinctive features are seven guideline-grounded Boolean flags derived from fraud-rule heuristics (e.g., `Liability_NoPolice`, `VeryEarlyClaimFlag`, `Sport_YoungDriver`). These align with investigator rules, improving both model performance and SHAP interpretability.

The pipeline produces two outputs from the engineered data: for XGBoost, pre-encoded categorical columns (`REPLACED_CATEGORICALS`) are dropped to avoid duplication, while for CatBoost they are retained, leveraging its native categorical handling.

2.4. Data Validation Schema

Before any preprocessing runs, the raw dataframe is validated against a Pandera schema [5] defined in `data_validation.py`. The schema enforces type, range, and value-set constraints for every column: Age must be an integer in [16, 100], Deductible must be one of {300, 400, 500, 700}, Month must be one of the 12 month abbreviations, FraudFound_P must be 0 or 1, and so on. A validation failure raises a `SchemaError` immediately and halts the pipeline. This is a small but important piece of production hygiene: it ensures that an upstream data change (a new month code, a new vehicle category, an unexpected null) is caught at ingest time rather than silently corrupting the model. The schema check is logged to MLflow as a pass/fail flag for every training run.

2.5. Dataset Composition Visualizations

Figure 1 shows the fraud-rate distribution on the primary automotive dataset. The bar on the right (the fraud-positive class) is roughly one-fifteenth the height of the bar on the left, which is the visual signature of the 5.99% prevalence rate the model has to learn against. This imbalance is the reason every metric reported in this paper emphasizes PR-AUC and Precision@k rather than headline accuracy.

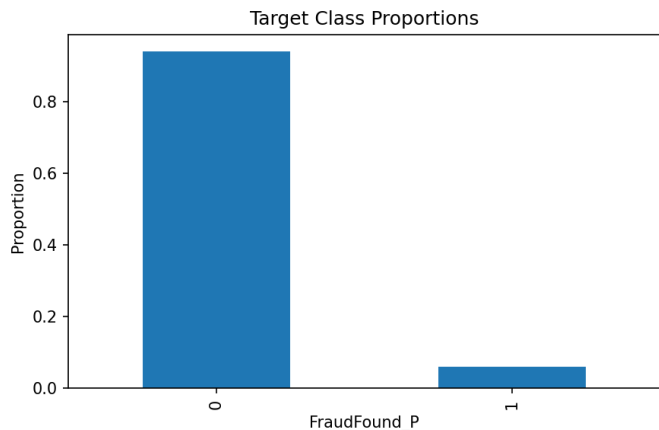


FIGURE 1: Fraud rate distribution in the automotive dataset. (0 = Legitimate, 1 = Fraud)

For the secondary Medicare dataset the corresponding fraud rate is 9.35% (506 fraudulent providers out of 5,410), giving a less severe but still meaningful imbalance ratio of 9.7:1. The cross-domain comparison and the fraud-rate figure for Medicare are presented in Section 5.

3. SYSTEM DESIGN AND ARCHITECTURE

3.1. Pipeline Overview

The core idea is to keep each stage of fraud triage simple, testable, and replaceable. A claim passes through a fixed sequence: ingest, validation, cleaning, feature engineering, scoring, calibration, explanation, retrieval-augmented brief generation, triage routing, and a human-facing dashboard. Each layer reads only the necessary inputs and produces a clean output for the next, with no direct coupling between stages. This allows

the pipeline to be improved incrementally without affecting other components.

This design improves both engineering quality and explainability. Each layer is independently tested, preventing local changes from breaking downstream stages. At the same time, all intermediate outputs are preserved, enabling full traceability: the final score can be linked back to features, those features to the processed input, and the input to the validated raw claim.

3.2. Data Validation and Preprocessing

Each claim first undergoes a schema validation step, where all columns are checked for expected types, ranges, and valid values. Any failure halts the pipeline, ensuring data issues are caught early.

After validation, the data is cleaned and split. Identifier-like columns are removed to prevent leakage, and sentinel values are imputed with appropriate replacements. The dataset is then partitioned into 80/10/10 stratified training, validation, and test sets, preserving class imbalance.

Preprocessing transformations, median imputation and standardization for numeric features, and mode imputation with one-hot encoding for categorical features are fit only on the training data and applied unchanged to validation and test sets, preventing data leakage during model training.

3.3. Modeling and Explainability

Three base models are trained: Logistic Regression as the baseline, and XGBoost and LightGBM as tree-based alternatives, all using the same features and train/validation split for comparability.

The strongest model is then improved through Bayesian hyperparameter tuning, with a second tuned tree model trained in parallel. Their predictions are combined using an out-of-fold stacking ensemble, and final outputs are calibrated to produce meaningful probabilities.

Predictions are explained using SHAP values, which provide both global feature importance and per-claim reason codes (e.g., “no police report”, “policy holder at fault”). These explanations are shown in the dashboard and used to generate triage briefs.

The same modeling and explanation pipeline is also applied to the Medicare dataset, keeping the methodology fixed while only changing the input features for cross-domain evaluation.

3.4. Retrieval-Augmented Brief Generation

A risk score and SHAP plot alone are insufficient for action, so the system generates a short retrieval-augmented triage brief [6] for each flagged claim, grounded in a curated fraud-guidelines corpus rather than model memory.

The corpus is processed once by splitting documents into passages, embedding them [7], and storing them in a persistent vector database [8] with metadata. The index is reused across runs without re-embedding.

At inference, a query built from the claim’s score, triage bucket, and top SHAP reason codes retrieves relevant guideline passages. These, along with claim metadata, are passed to GPT-4o-mini to generate a structured brief with citations. If no API

is available, a template-based brief is generated instead. In both cases, retrieved passages and similarity scores are stored, ensuring full traceability to the underlying guidelines.

3.5. Dashboard, Deployment, and Experiment Tracking

The user-facing layer is an interactive dashboard [9] covering key operational views: a KPI summary, a queue of flagged claims, per-claim drill-down with SHAP explanations and RAG briefs, real-time claim scoring, fairness analysis, drift monitoring, temporal performance, and dataset comparison. Each page includes a collapsible panel explaining metrics in plain language for investigator use.

Figure 2 shows the Summary Dashboard, which is the primary landing page.

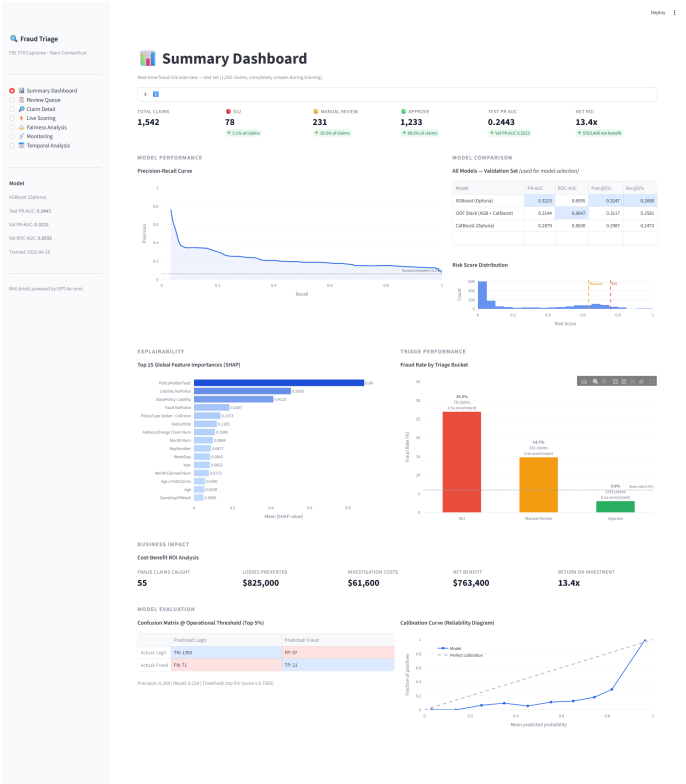


FIGURE 2: Summary Dashboard showing KPIs, precision–recall curve, model comparison, SHAP importance, triage performance, ROI breakdown, and calibration.

The system is containerised for consistent execution across local and cloud environments, with separate full and lightweight images. A CI/CD pipeline rebuilds and deploys updates automatically, running tests on each push to prevent faulty deployments.

All training runs are tracked end-to-end, logging parameters, dataset statistics, performance metrics, business outcomes, runtime, and model artifacts. Runs are hierarchically organized to capture both the full pipeline and individual stages, enabling comparison and providing an audit trail [10].

4. METHODOLOGY AND MODEL DEVELOPMENT

4.1. Model Training Setup and Procedure

Three base classifiers were trained on the engineered features: logistic regression as a linear baseline, and XGBoost[11]

and LightGBM [12] as gradient-boosted models. All were trained on the same 12,336-claim training set and evaluated on the same 1,542-claim validation set for direct comparability.

Class imbalance was handled within the objective by assigning XGBoost and LightGBM a positive-class weight equal to the inverse fraud rate, focusing learning on the minority class without altering the data distribution.

The strongest base model was then refined through Bayesian hyperparameter optimisation using a Tree-structured Parzen Estimator (TPE) search [13]. The same tuning process was applied to a CatBoost [14] model in parallel.

The tuned models were combined using an out-of-fold stacking ensemble, where predictions from XGBoost and CatBoost across three folds served as inputs to a logistic-regression meta-learner. The meta-learner remains intentionally simple to avoid overfitting and learns an optimal linear combination of model outputs.

4.2. Calibration and Triage Routing

Tree-based model outputs are not inherently well-calibrated probabilities. A claim with a raw score of 0.9 is not necessarily ten times more likely to be fraudulent than one with a score of 0.09. Since downstream decisions depend on absolute thresholds, isotonic calibration was applied to transform model outputs into meaningful probabilities.

The calibrated scores are mapped into three triage buckets. The top five percent of claims are routed to the Special Investigations Unit (SIU) for full investigation. The next fifteen percent are assigned to manual review, and the remaining eighty percent are auto-approved. These thresholds are chosen to reflect realistic SIU capacity rather than to optimise a single statistical metric. A system that flags too many claims as high-risk may perform well statistically but is operationally infeasible.

4.3. Evaluation Metrics

Evaluation is driven by class imbalance. With a 5.99% fraud rate, accuracy is not meaningful, so PR-AUC is used as the primary metric for model selection and final test evaluation. ROC-AUC is also reported for comparison, though it is less informative in imbalanced settings. Precision@5% and Recall@5% measure how well fraud is concentrated and captured within the SIU bucket.

Business metrics are computed alongside statistical performance by assigning fixed values to detected fraud and investigation costs, yielding net benefit and ROI.

Fairness is evaluated using the 80% disparate-impact rule, comparing group-wise flag rates, with groups having fewer than 30 samples excluded to ensure stability.

4.4. Retrieval-Augmented Triage Brief Generation

The final stage generates a short triage brief for each flagged claim, determining whether the system is operationally useful or merely statistically informative.

The guideline corpus is processed once offline: documents are split into passages, embedded as vectors, and stored in a

persistent database with source metadata. The index is reused across runs, so embedding cost is incurred only once.

At inference, a query built from the claim’s calibrated risk score, triage bucket, and top SHAP reason codes retrieves the most relevant guideline passages via vector similarity. These passages, along with claim metadata, are passed to GPT-4o-mini to generate a structured brief with explicit citations.

If no API is available, the system falls back to a template-based generator using the same retrieved passages. Both modes produce actionable outputs, with GPT-4o-mini providing more natural phrasing. Retrieved passages and similarity scores are always preserved, ensuring full traceability to the underlying guideline evidence.

Together, this design balances predictive performance, interpretability, and operational usability, aligning model outputs with real-world investigative workflows.

5. RESULTS AND DISCUSSION

5.1. Model Performance

The three base models established a clear baseline, with XGBoost performing best on validation precision–recall, followed by LightGBM and logistic regression. After feature engineering and Bayesian tuning, XGBoost achieved a validation PR-AUC of 0.3223, about 28% higher than its baseline. CatBoost reached 0.2879, and their out-of-fold stacking ensemble reached 0.3144. Tuned XGBoost was selected as the final model due to its highest validation performance and simpler deployment.

The held-out test set was evaluated once for the selected model, yielding a PR-AUC of 0.2443 and Precision@5% of 0.3247. Table 2 summarizes model comparisons, and Figure 3 shows the test-set precision–recall curve.

TABLE 2: Validation performance of the base models and the improved (tuned) models on the automotive dataset.

Model	Val PR-AUC	Val ROC-AUC	Prec. @5%	Test PR-AUC
Logistic Regression (baseline)	0.1677	0.8108	0.1818	—
LightGBM (baseline)	0.1846	0.8097	0.2208	—
XGBoost (baseline)	0.2518	0.8346	0.2857	—
CatBoost (Optuna)	0.2879	0.8630	0.2987	—
OOF Stacking (XGB + CatBoost)	0.3144	0.8647	0.3117	—
XGBoost (Optuna, selected)	0.3223	0.8595	0.3247	0.2443

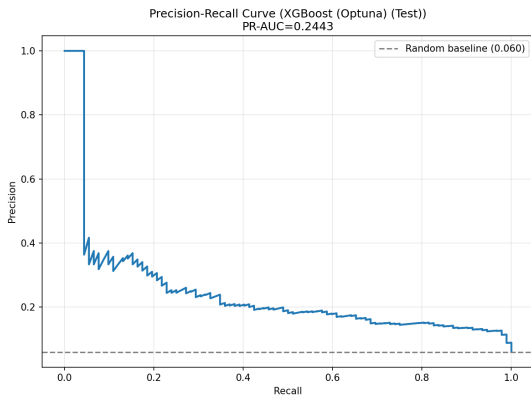


FIGURE 3: Precision–recall curve of the selected XGBoost (Optuna) model on the held-out test set (PR-AUC = 0.2443). The dotted line shows the random baseline at the 5.99% fraud rate.

5.2. Triage Bucket Performance and ROI

The numbers that matter operationally are not the raw precision–recall scores but how well the calibrated probabilities concentrate fraud at the top of the ranking. Table 3 shows the per-bucket fraud rates on the test partition. The top-5% SIU bucket contains 32.5 percent fraud, which is a 5.4× enrichment over the underlying 5.99 percent base rate. The next-15% manual-review bucket contains 16.0 percent fraud, a 2.7× enrichment. The auto-approve bucket, which absorbs the bulk of the volume, has a residual fraud rate of just 2.5 percent.

TABLE 3: Triage bucket performance on the held-out test set. Enrichment is computed against the 5.99% base fraud rate.

Triage Bucket	Claims	Fraud Rate	Enrich. vs. Base
SIU Escalation (top 5%)	77	32.5%	5.4×
Manual Review (next 15%)	231	16.0%	2.7×
Auto-Approve (remaining 80%)	1,234	2.5%	0.4×

These numbers translate directly into business value. ROI is computed on the held-out test set (1,542 claims, never seen during training or model selection) to ensure an honest, unbiased estimate. Using cost assumptions of \$15,000 average fraud loss, \$500 per SIU investigation, and \$100 per manual review, the system catches 55 fraudulent claims, prevents an estimated \$825,000 in losses, incurs \$61,600 in investigation costs, and yields a net benefit of \$763,400 — a return of 13.4× on every dollar spent investigating flagged claims. Earlier validation-set ROI figures (computed during model selection) are higher because the model was tuned on that partition; the test-set figure reported here is the honest production estimate.

5.3. Explainability and SHAP Findings

Globally, the model relies on a small handful of features to do most of its work. Table 4 lists the top five features by mean absolute SHAP [15] value from the selected XGBoost Optuna model evaluated on the held-out test set. The dominant feature is PolicyHolderFault, an engineered binary indicator capturing whether the policyholder was at fault, which directly reflects claim responsibility and strongly correlates with fraudulent behavior.

The second most important feature is Liability_NoPolice, one of the seven engineered guideline-grounded flags — the conjunction of liability-only coverage and the absence of a police report — which the internal fraud-guideline documents explicitly flag as a high-risk configuration. This feature is particularly important because it encodes a domain rule that would otherwise require manual investigation, allowing the model to capture investigator knowledge directly.

Fault_NoPolice (rank 4) is also an engineered interaction feature, reinforcing the same risk pattern by combining fault attribution with missing documentation. The presence of multiple related engineered features among the top ranks indicates that the model is consistently identifying and leveraging these high-risk combinations rather than relying on isolated signals.

Together, three of the top five features are engineered, mean-

ing that engineered features collectively account for the majority of the model’s explanatory power. This highlights the importance of domain-informed feature construction, as the original raw schema alone would not have produced this kind of structured and interpretable ranking of risk factors.

TABLE 4: Top five global features by mean absolute SHAP value (XGBoost Optuna, test set). Engineered features dominate the ranking.

Rank	Feature	Mean SHAP	Source
1	PolicyHolderFault	0.880	Engineered
2	Liability_NoPolice	0.504	Engineered (guideline-grounded)
3	BasePolicy_Liability	0.412	Raw (OHE)
4	Fault_NoPolice	0.184	Engineered (interaction)
5	PolicyType_Sedan - Collision	0.137	Raw (OHE)

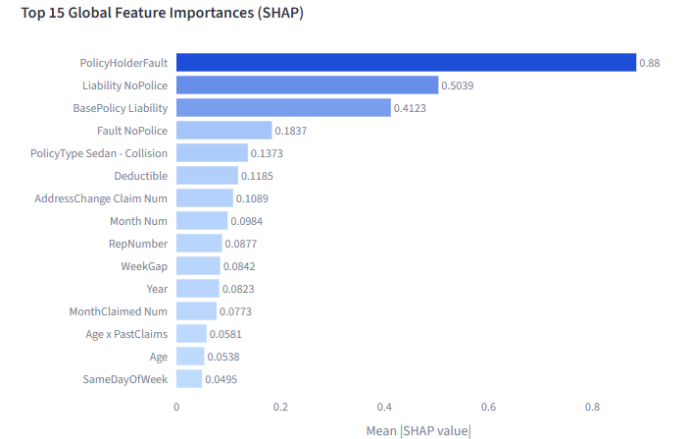


FIGURE 4: Global SHAP feature importance ranking (XGBoost Optuna, test set). Top features match Table 4.

At the per-claim level, the top three SHAP contributions for each flagged claim are translated into short reason codes that appear on the dashboard’s Claim Detail page. These reason codes are not just an explainability artefact: they are also passed into the prompt that builds the retrieval-augmented brief, which is what binds the model’s output to the guideline corpus.

5.4. RAG Triage Brief Quality

The triage brief is the primary artefact for investigators and is therefore evaluated qualitatively rather than through a single metric. Across the test set, every flagged claim received a brief grounded in at least one of the four fraud-guideline documents, ensuring outputs rely on the guideline corpus rather than model memory.

When the language-model API was available, briefs were generated in natural prose with explicit citations; otherwise, the system produced a template-based brief using the same retrieved passages. Both modes were found acceptable, with the language-model version offering better readability and the template version providing more structured but equally faithful outputs. Figure 5

shows the per-claim drill-down view combining the risk score, SHAP reason codes, and generated brief.

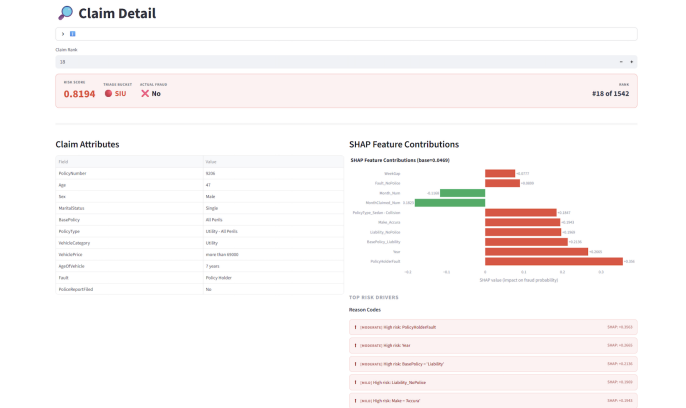


FIGURE 5: Claim Detail page showing risk score, SHAP reason codes, and RAG triage brief.

5.5. Fairness Audit

The fairness audit applied the 80-percent disparate-impact rule across age, sex, and marital status. Two dimensions passed, but the sex dimension produced a ratio of 0.574 (below the 0.80 threshold), with male policyholders flagged at ~22% and females at ~12%. This does not necessarily indicate bias, as underlying fraud rates may differ, but it highlights the need to examine potential proxy features (e.g., vehicle category or agent type) before deployment. Figure 6 shows the dashboard view of these ratios.

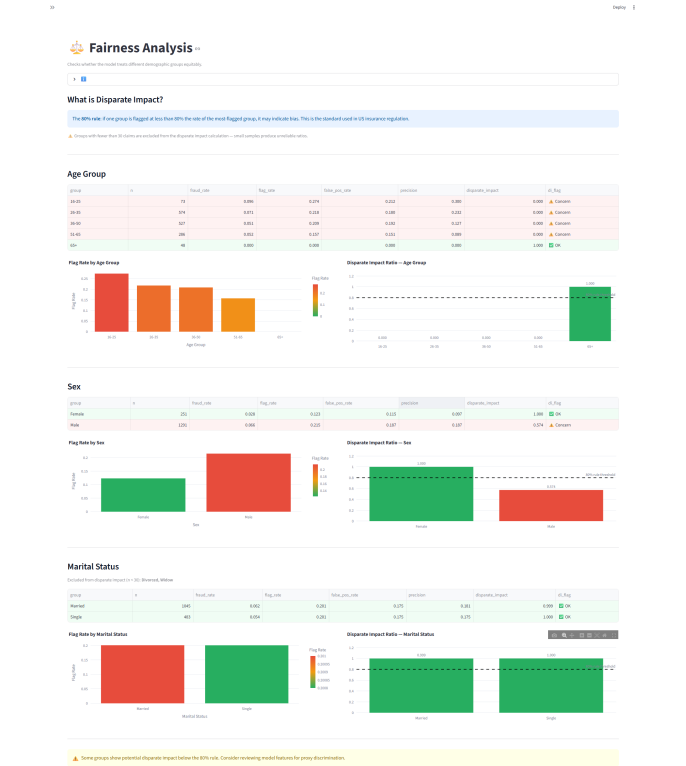


FIGURE 6: Fairness Analysis page showing group flag rates, fraud rates, and disparate-impact ratios.

TABLE 5: Cross-domain comparison between the automotive insurance and Medicare provider models. The methodology is held fixed; only the input features differ.

Metric	Automotive	Medicare
Records (primary unit)	15,420 claims	5,410 providers
Fraud rate	5.99%	9.35%
Imbalance ratio	15.7:1	9.7:1
Validation PR-AUC	0.3223	0.7874
Test PR-AUC	0.2443	0.6869
SIU enrichment	5.4×	8.9×

5.6. Drift Monitoring and Temporal Analysis

Fraud models can degrade as patterns and data distributions change. To simulate monitoring, the test set was split into six time-ordered batches and evaluated using summary statistics and Kolmogorov–Smirnov tests. Risk scores and fraud rates remained stable, with no significant drift detected, as expected for a held-out test set.

Month-by-month analysis showed similar stability, with no major performance collapse. These monitoring views are implemented in the dashboard and can be extended to live production data.

5.7. Cross-Domain Validation: Medicare

To test whether the methodology generalises beyond automotive insurance, the same pipeline Bayesian-tuned XGBoost, SHAP explanations, calibrated triage buckets was retrained on the Medicare provider dataset. Only the input features changed; everything else was held fixed. Table 5 shows the cross-domain comparison. The Medicare model reaches a Test PR-AUC of 0.6869 and an SIU enrichment of 8.9×, both substantially higher than the automotive numbers.

The headline difference is not because the methodology works better on one dataset than the other. It reflects the underlying nature of the two fraud problems. Medicare fraud is committed at the provider level by systematic over-billing across hundreds of claims, which means a single aggregate feature such as total reimbursement separates fraudulent from legitimate providers cleanly. Automotive fraud is committed one claim at a time and is engineered to look like a legitimate accident, which is structurally a much harder signal to detect. The top SHAP features confirm this difference: the automotive model leans on claim-level characteristics such as fault and police-report status, while the Medicare model leans on aggregate billing volumes. The methodology engineer features, tune with Bayesian search, calibrate, route into triage buckets transfers; the features themselves do not. For instance, the top Medicare SHAP feature TotalReimbursed (mean |SHAP| = 1.131) alone cleanly separates fraudulent from legitimate providers, a signal with no equivalent in the automotive domain where the top feature PolicyHolderFault (mean |SHAP| = 0.880) reflects claim-level behavior rather than billing volume.

5.8. Data-Leakage Ablation

A short ablation captures the importance of correctly excluding the row-identifier column from the feature set. Table 6 reports

the same metrics with and without the column included. Including it produces precision–recall scores that look state-of-the-art but are entirely artefacts of the model memorising the row identity. Excluding it produces the lower but honest numbers reported throughout this paper.

TABLE 6: Data-leakage ablation. Including the row-identifier column inflates precision–recall scores by more than three-fold; the corrected pipeline reports honest numbers.

Configuration	Val PR-AUC	Test PR-AUC
With row identifier (leakage)	0.7166	0.7926
Without row identifier (corrected)	0.2518	0.1951

6. DISCUSSION AND LIMITATIONS

The system is designed around interpretability and operational usability. Each prediction is traceable from the final score to SHAP reason codes, engineered features, and the original validated claim. Triage briefs are grounded in cited guideline language rather than model memory, and all training runs are logged with parameters, metrics, and artifacts to ensure auditability. The modular design allows independent improvement of components such as feature engineering, modeling, and brief generation.

There are, however, key limitations. Automotive fraud is harder to detect than provider-level fraud, as each claim mimics a legitimate event, creating a natural ceiling on performance; the test PR-AUC of 0.2443 should be interpreted in this context. The fairness audit also found a disparate-impact ratio for sex below 0.80, indicating the need to examine potential proxy variables before deployment. Additionally, drift monitoring is limited by the static test set and becomes more meaningful with real production data.

Overall, the system is intended as decision support, not automation. The calibrated scores, SHAP explanations, and RAG briefs assist investigators rather than replace them. Effective deployment requires a human-in-the-loop approach with clear triage thresholds—high-risk claims escalated to SIU, medium-risk reviewed, and low-risk sampled—under which the reported ROI estimates are valid.

7. RECOMMENDATIONS

The system design suggests several deployment recommendations. Triage thresholds should remain configurable rather than fixed, allowing adjustment based on investigator capacity, claim values, and costs, supported by calibrated probabilities.

The fairness result for sex requires further investigation, as the disparate-impact ratio falls below 0.80. Potential proxy features (e.g., vehicle category or agent type) should be examined and remediated if necessary.

Drift monitoring should run continuously on live data, extending batch-level checks to rolling windows with alerts for distribution shifts. Investigator feedback should also be logged and incorporated into retraining for continuous improvement.

Finally, the system should operate with a human-in-the-loop design. Model outputs, SHAP explanations, and RAG briefs are intended to support, not replace, investigators, with clear triage thresholds guiding escalation, review, and auditing decisions.

8. CONCLUSION

This project developed an end-to-end fraud triage system for automotive insurance claims and validated the same methodology on a Medicare provider-fraud dataset to demonstrate cross-domain generalisation. The pipeline integrates schema validation, guideline-informed feature engineering, Bayesian-tuned XGBoost modeling, probability calibration, SHAP-based explainability, and retrieval-augmented brief generation into a single cohesive workflow.

On the automotive dataset, the selected model concentrates 32.5% fraud in the top-five-percent SIU bucket — a 5.4× enrichment over the base rate — yielding an estimated net benefit of \$763,400 and a 13.4× return on investment. When applied to the Medicare dataset, the same methodology achieves a Test PR-AUC of 0.6869 and an 8.9× SIU enrichment, confirming that the framework generalises across domains despite differences in feature structure and aggregation level.

Beyond predictive performance, the system is designed for real-world deployment. It is supported by a containerised dashboard, CI/CD-based deployment pipeline, experiment tracking, automated testing, and fairness auditing using the 80% disparate-impact rule. Together, these components position the system not as a standalone model, but as a complete, interpretable, and operationally viable decision-support framework for fraud triage in insurance workflows.

ACKNOWLEDGEMENTS

We thank our project advisor, Dr. Joshua Loughman and teaching assistant, Erfan Ziad for their guidance and feedback throughout this project. We also acknowledge the FSE 570 Capstone teaching team at Arizona State University for providing the structure and support that enabled this work, and the Kaggle community for making the datasets publicly available. The full codebase and live demo are available at <https://github.com/asing758-vs/insurance-claims-risk-triage> and <https://huggingface.co/spaces/Mriks/fraud-triage>.

REFERENCES

- [1] Federal Bureau of Investigation, “Insurance Fraud — Common Fraud Schemes,” Available: <https://www.fbi.gov/scams-and-safety/common-frauds-and-scams/insurance-fraud>, Accessed: Apr. 29, 2026.
- [2] Kaggle, “Vehicle Claim Fraud Detection Dataset,” Available: <https://www.kaggle.com/datasets/shivamb/vehicle-claim-fraud-detection>, Accessed: Apr. 29, 2026.
- [3] Kaggle, “Healthcare Provider Fraud Detection Analysis Dataset,” Available: <https://www.kaggle.com/datasets/rohitro/healthcare-provider-fraud-detection-analysis>, Accessed: Apr. 29, 2026.
- [4] Pedregosa, F., et al., 2011, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, Vol. 12, pp. 2825–2830.
- [5] Pandera Project, “Pandera: A Statistical Data Validation Toolkit for Pandas,” Available: <https://pandera.readthedocs.io>, Accessed: Apr. 29, 2026.
- [6] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al., 2020, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*.
- [7] Reimers, N., and Gurevych, I., 2019, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *Proceedings of EMNLP*.
- [8] Chroma, “Chroma: The AI-native Open-Source Embedding Database,” Available: <https://www.trychroma.com>, Accessed: Apr. 29, 2026.
- [9] Streamlit, “Streamlit: A Faster Way to Build and Share Data Apps,” Available: <https://streamlit.io>, Accessed: Apr. 29, 2026.
- [10] MLflow Project, “MLflow: An Open-Source Platform for the Machine Learning Lifecycle,” Available: <https://mlflow.org>, Accessed: Apr. 29, 2026.
- [11] Chen, T., and Guestrin, C., 2016, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- [12] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y., 2017, “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems (NeurIPS)*.
- [13] Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M., 2019, “Optuna: A Next-Generation Hyperparameter Optimization Framework,” *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- [14] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A., 2018, “CatBoost: Unbiased Boosting with Categorical Features,” *Advances in Neural Information Processing Systems (NeurIPS)*.
- [15] Lundberg, S. M., and Lee, S.-I., 2017, “A Unified Approach to Interpreting Model Predictions,” *Advances in Neural Information Processing Systems (NeurIPS)*.